COS214PRAC4 PDF DOCUMENT

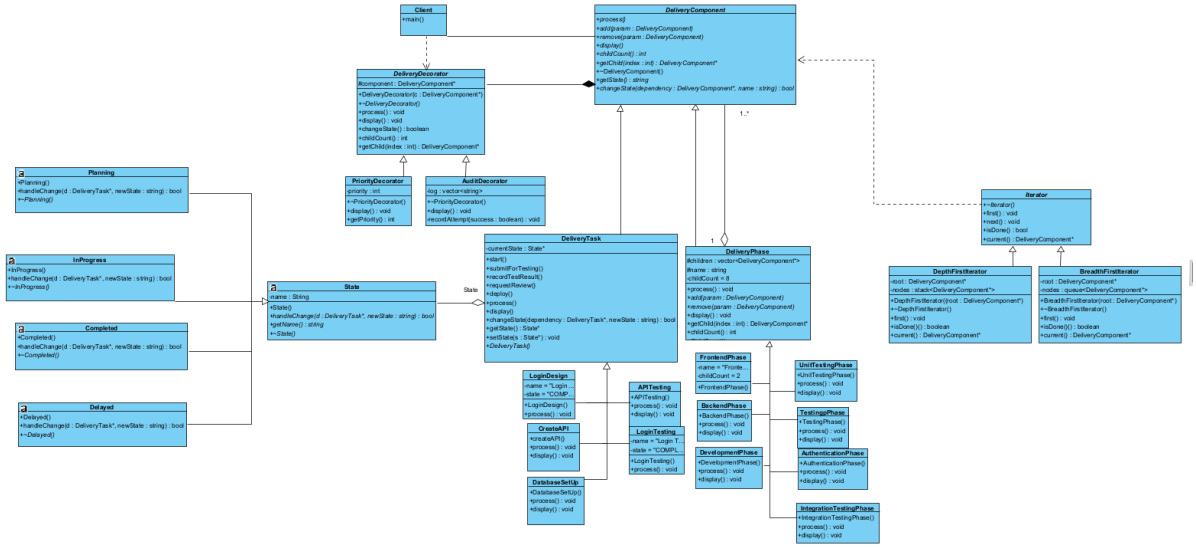
Team members:

Lesedi Shelile	u25110455
Taya Govender	u24697274
Shakir Alexander	u25122208

Taskforge: Software delivery system

TaskForge is a software delivery system that models the process required to deliver a software product. It structures work into a recursive hierarchy of work groups and individual tasks, enabling complex projects to be represented and managed clearly. Individual tasks progress through a defined lifecycle as work and outcomes are recorded, while additional responsibilities (such as high-priority handling, specialised testing, or extra review) can be dynamically attached when required. The system also provides different ways to traverse and inspect the work hierarchy without exposing its underlying structure.

UML



Design rationale

Our design uses a Composite to represent the software delivery hierarchy. There are many phases one must go through while preparing an object for delivery, and every phase has tasks associated with it. DeliveryComponent is the common abstraction for both DeliveryPhase groups and individual DeliveryTask objects. DeliveryPhase owns a vector<DeliveryComponent*>, allowing phases to contain both tasks and other phases and therefore form the required nested hierarchy.

For the Iterator design patter, we have a DepthFirstIterator and a BreadthFirstIterator which both traverse the DeliveryPhase hierarchy independently. The traversal logic is kept inside the iterator classes rather than exposing vector<DeliveryPhase*> children to Main.cpp, satisfying the requirement that clients do not access the internal container directly.

For the State design pattern, we have a DeliveryTask class which delegates state changes to its current state object (Planning, InProgress, Delayed, or Completed). This keeps transition rules inside the relevant state classes; for example, a task in Planning cannot move directly to Completed, while InProgress can transition to Completed or Delayed. Dependencies are passed to handleChange () only for states where they affect a transition. Planning and Delayed check the dependency before allowing INPROGRESS; InProgress and Completed do not need the dependency because their transitions are determined solely by their own current state.

For the Decorator design pattern, we have a PriorityDecorator and AuditDecorator wrap a DeliveryComponent rather than modifying DeliveryPhase or DeliveryTask. This allows responsibilities such as priority and auditing to be added at runtime while the wrapped object remains usable through the same DeliveryComponent interface.

In terms of ownership, DeliveryPhase owns its child relationships and stores them as DeliveryComponent*, while Main.cpp is responsible for creating and destroying the objects. This gives each dynamically allocated object one clear destruction point and avoids duplicating ownership between the composite and client.

GOF participants

1. Composite

Participant Name	**Class**	**Description**
Component	DeliveryComponent	Common interface for phases and tasks.
Composite	DeliveryPhase	Can contain multiple DeliveryComponent objects.
Concrete Composite	FrontendPhase	Contains frontend-related tasks.
Concrete Composite	BackendPhase	Contains backend-related tasks.
Concrete Composite	TestingPhase	Can contain testing sub-phases.
Leaf	DeliveryTask	Represents an individual task that cannot contain children.

Concrete Leaf	LoginDesign, LoginTesting, DatabaseSetUp, CreateAPI, APITesting	Concrete TaskForge tasks.

2. State

Participant Name	Class	Description
Context	DeliveryTask	Maintains the current state of a task and delegates state changes to the current State object.
State	State	Defines the interface for handling task state changes.
Concrete State	Planning	Handles transitions while a task is in the PLANNING state.
Concrete State	InProgress	Handles transitions while a task is INPROGRESS.
Concrete State	Delayed	Handles transitions while a task is DELAYED.
Concrete State	Completed	Represents a completed task and prevents invalid further transitions.

3. Iterator

Participant Name	Class	Description
Iterator	Iterator	Defines the interface for traversing TaskForge components.
Concrete Iterator	DepthFirstIterator	Traverses the hierarchy depth-first using a stack.
Concrete Iterator	BreadthFirstIterator	Traverses the hierarchy breadth-first using a queue.

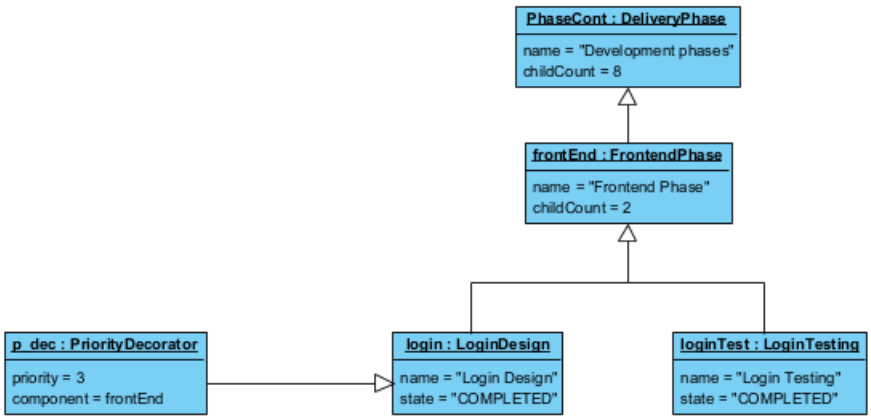
Traversed Component	DeliveryComponent	Provides child-access operations used by the iterators to traverse the hierarchy.

4. Decorator

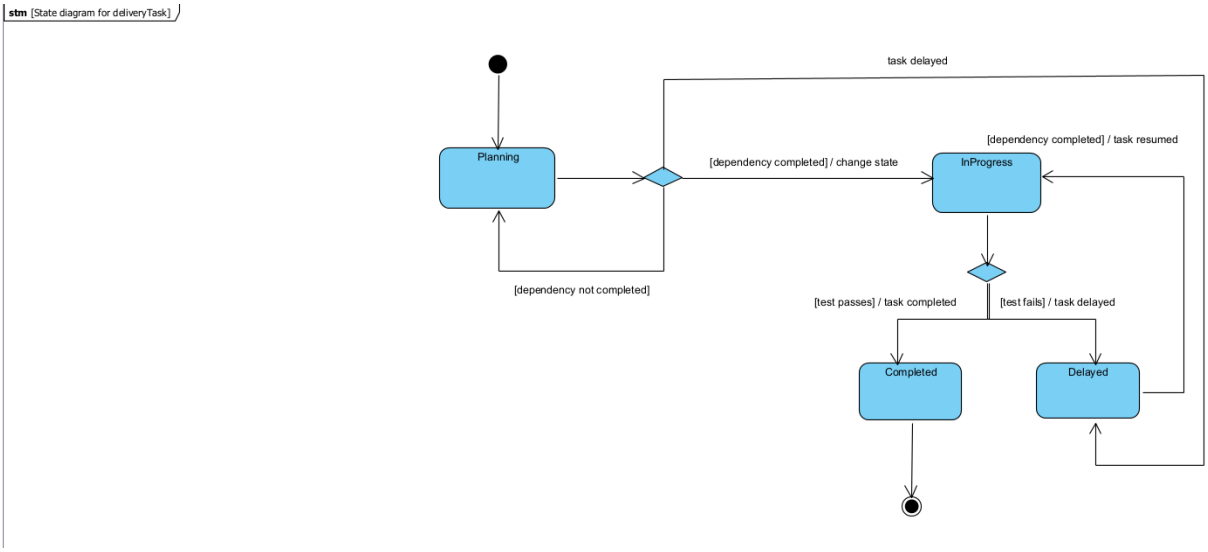
Participant Name	**Class**	**Description**
Component	DeliveryComponent	Defines the interface shared by components and decorators.
Concrete Component	DeliveryPhase / FrontendPhase	Provides the original component being decorated.
Decorator	DeliveryDecorator	Wraps a DeliveryComponent and forwards its operations.
Concrete Decorator	PriorityDecorator	Adds a priority value to the wrapped component.
Concrete Decorator	AuditDecorator	Adds logging of state-change attempts to the wrapped component.

COS214PRAC4 PDF DOCUMENT

UML OBJECT DIAGRAM

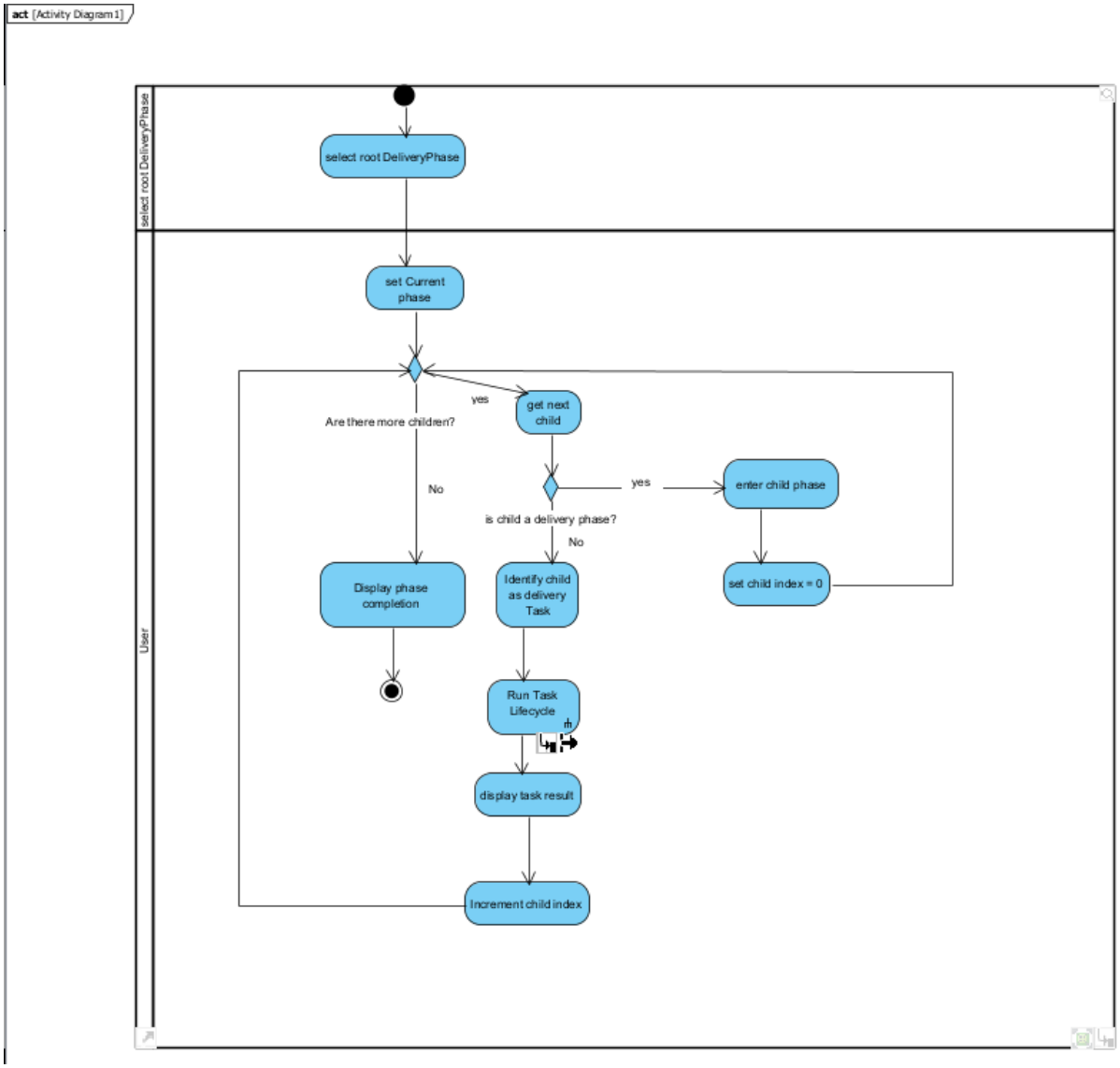


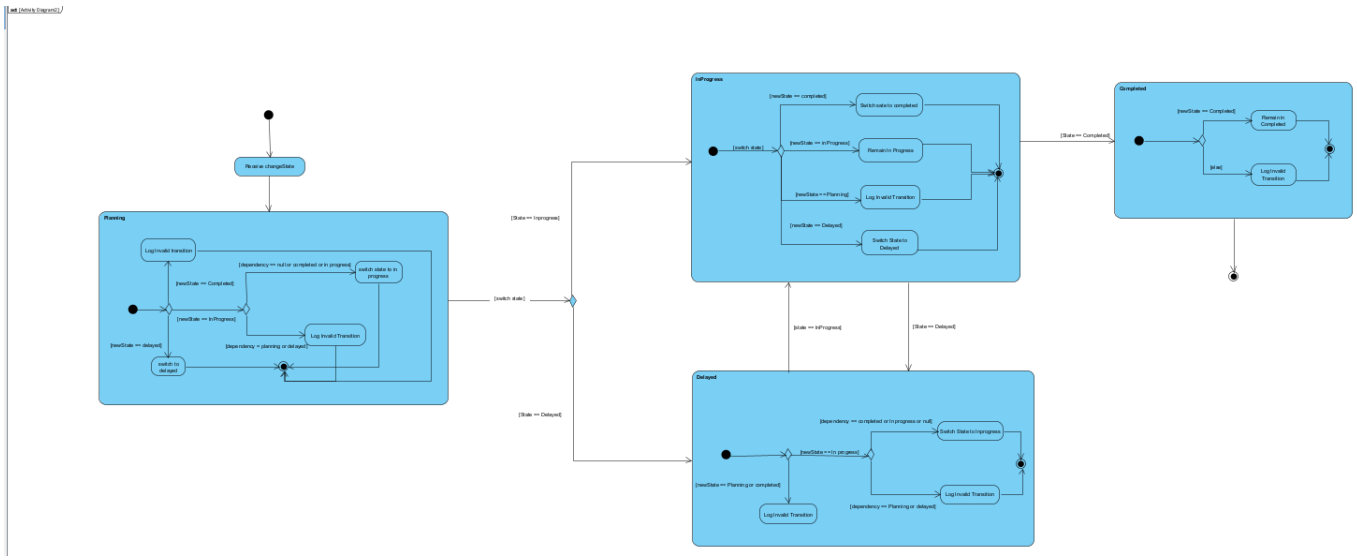
UML STATE DIAGRAM

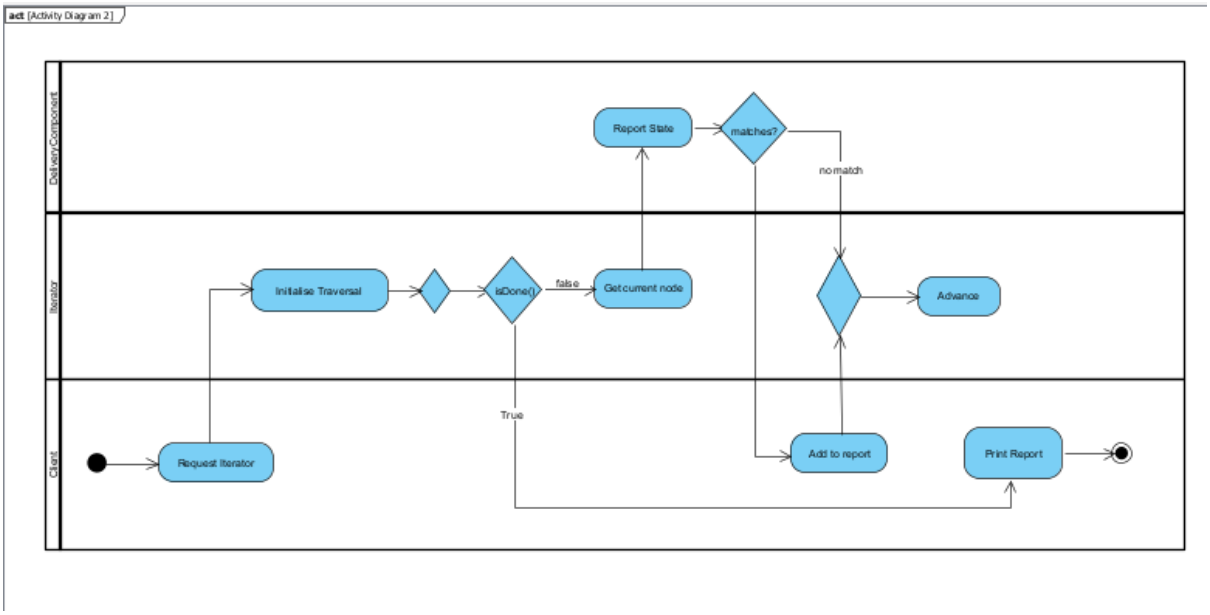


UML ACTIVITY DIAGRAMS

COS214PRAC4 PDF DOCUMENT







GDB investigation and Valgrind check

During the testing of the TaskForge system GDB was used to assist with diagnosing any bugs that were encountered. One such bug was a segmentation fault due to a function trying to access an attribute of a class that had no instantiated value. GDB was used to set a breakpoint at the function that caused the segment fault. This was rectified by adding an initial value to the attribute. Evidence of GDB usage:

```
Breakpoint 1, main () at Main.cpp:134
134         login->changeState(NULL, "PLANNING");
(gdb) n

Program received signal SIGSEGV, Segmentation fault.
DeliveryTask::changeState (this=0x55555557ba80, dependency=0x0, newState="PLANNING") at DeliveryTask.cpp:57
57          return this->currentState->handleChange(dependency, newState);
(gdb) backtrace
#0  DeliveryTask::changeState (this=0x55555557ba80, dependency=0x0, newState="PLANNING") at DeliveryTask.cpp:57
#1  0x000055555556044b in main () at Main.cpp:134
(gdb) backtrace
#0  DeliveryTask::changeState (this=0x55555557ba80, dependency=0x0, newState="PLANNING") at DeliveryTask.cpp:57
#1  0x000055555556044b in main () at Main.cpp:134
(gdb)
```

The second bug that was encountered was an ownership double free bug, thus causing a segmentation fault. A class destructor was attempting to delete an object it did not have ownership over, thus when both destructors were called the object was deleted twice. This was diagnosed by back tracing after the program crashed and resolved by removing the delete statement from the class that did not own the object. Evidence of GDB usage:

```
taya@LAPTOP-RU7E7AEH:~/COS214PRAC4$ gdb ./main
(gdb) r

Program received signal SIGSEGV, Segmentation fault.
0x000055555555ccbc in DeliveryDecorator::~DeliveryDecorator (this=0x55555557f370, __in_chrg=<optimized out>)
    at DeliveryDecorator.cpp:8
8           delete this->component;
(gdb) backtrace
#0  0x000055555555ccbc in DeliveryDecorator::~DeliveryDecorator (this=0x55555557f370, __in_chrg=<optimized out>)
    at DeliveryDecorator.cpp:8
#1  0x00005555555587e2 in AuditDecorator::~AuditDecorator (this=0x55555557f370, __in_chrg=<optimized out>)
    at AuditDecorator.cpp:12
#2  0x0000555555558802 in AuditDecorator::~AuditDecorator (this=0x55555557f370, __in_chrg=<optimized out>)
    at AuditDecorator.cpp:12
#3  0x00005555555560fec in main () at Main.cpp:212
(gdb)
```

After the Testing was completed and any bugs were rectified Valgrind was used to ensure proper memory management. Due to some of the earlier bugs and poor memory management there was memory leaks. After conducting a valgrind check the leaks were removed and correct memory management was seen throughout the system. Evidence of Valgrind usage:

```
taya@LAPTOP-RU7E7AEH:~/COS214PRAC4$ valgrind --leak-check=full --track-origins=yes ./main
==313048== 384 (64 direct, 320 indirect) bytes in 1 blocks are definitely lost in loss record 25 of 25
==313048==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==313048==    by 0x113D07: main (Main.cpp:46)
==313048==
==313048== LEAK SUMMARY:
==313048==    definitely lost: 448 bytes in 8 blocks
==313048==    indirectly lost: 716 bytes in 17 blocks
==313048==      possibly lost: 0 bytes in 0 blocks
==313048==    still reachable: 0 bytes in 0 blocks
==313048==         suppressed: 0 bytes in 0 blocks
==313048==
==313048== For lists of detected and suppressed errors, rerun with: -s
==313048== ERROR SUMMARY: 8 errors from 8 contexts (suppressed: 0 from 0)
taya@LAPTOP-RU7E7AEH:~/COS214PRAC4$
```

```
taya@LAPTOP-RU7E7AEH:~/COS214PRAC4$ valgrind --leak-check=full --track-origins=yes ./main
==314928== 95 (64 direct, 31 indirect) bytes in 1 blocks are definitely lost in loss record 4 of 4
==314928==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==314928==    by 0x113D8F: main (Main.cpp:50)
==314928==
==314928== LEAK SUMMARY:
==314928==    definitely lost: 128 bytes in 3 blocks
==314928==    indirectly lost: 31 bytes in 1 blocks
==314928==      possibly lost: 0 bytes in 0 blocks
==314928==    still reachable: 0 bytes in 0 blocks
==314928==         suppressed: 0 bytes in 0 blocks
==314928==
==314928== For lists of detected and suppressed errors, rerun with: -s
==314928== ERROR SUMMARY: 3 errors from 3 contexts (suppressed: 0 from 0)
taya@LAPTOP-RU7E7AEH:~/COS214PRAC4$
```

```
==456655==
==456655== HEAP SUMMARY:
==456655==     in use at exit: 0 bytes in 0 blocks
==456655==   total heap usage: 63 allocs, 63 frees, 78,255 bytes allocated
==456655==
==456655== All heap blocks were freed -- no leaks are possible
==456655==
==456655== For lists of detected and suppressed errors, rerun with: -s
==456655== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
taya@LAPTOP-RU7E7AEH:~/COS214PRAC4$
```

Work split and GitHub integration

The TaskForge system was divided amongst three team members according to the major components of the system. Team member 1 was responsible for the composite implementation, drawing and mapping required composite structure in the UML as well as the object and activity diagram associated with the pattern. Furthermore, team member 1 was responsible for the docker file creation and testing as well as all the GitHub reflection tasks (repository link, workflow reflection and team contribution statement). Team member 2 was responsible for the state design pattern implementation of the project, drawing and mapping the required state GoF participants in the UML and PDF as well as drawing the state diagram and lifecycle activity diagram. They were also responsible for the GDB investigation and Valgrind final check. Team member 3 was responsible for the Iterator and decorator design pattern implementation, the final activity diagram as well as the ReadMe, main.cpp and Makefile. The work was integrated through the shared GitHub repository. Branches were created for each feature. When implementation of each feature was finished, each team member pushed their contributions to origin/main. Integration was performed incrementally rather than waiting until the end of the project, allowing compilation and functionality issues to be identified and resolved as development progressed. These activities also helped ensure that the final TaskForge implementation, tests and documentation were developed collaboratively rather than as a single final upload.

GitHub link - https://github.com/LesediShelile/COS214PRAC4.git